МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Лабораторная работа №8

Аннотации. Stream API

Выполнил: студент группы ___БВТ2402___
Голиков Михаил Вячеславович
Руководитель: _Мосева Марина Сергеевна_

Москва, 2077

Цель работы

Целью лабораторной работы является Разработка многопоточной системы обработки данных с использованием Stream API и пользовательских аннотаций для создания гибкого и расширяемого конвейера обработки.

Индивидуальное задание

- Аннотация @DataProcessor — создать для маркировки методов обработки
- Класс DataManager с методами:
- registerDataProcessor() — регистрация обработчиков с аннотацией
- loadData() — загрузка из источника (файл/БД/сеть)
- processData() — многопоточная обработка через Stream API
- saveData() — сохранение результатов
- Обработчики данных — несколько классов с методами, помеченными @DataProcessor (фильтрация, трансформация, агрегация)
- Многопоточность — использовать java.util.concurrent для параллельной обработки
- Тестирование — полный цикл: загрузка → обработка → сохранение

Основная часть

В процессе выполнения лабораторной работы были реализованы три программы, демонстрирующие различные подходы к многопоточному программированию:

1. **ArraySum** – программа для вычисления максимального элемента в большом массиве с использованием ExecutorService. Массив разделяется на 4 части, каждая из которых обрабатывается в отдельном потоке. Результаты объединяются в главном потоке.

2. **MatrixCalc** – программа для поиска максимального элемента в матрице с использованием наследования от класса Thread. Каждый поток обрабатывает отдельную строку матрицы, после чего результаты сравниваются в главном потоке.
3. **WarehouseForkJoin** – программа для моделирования работы грузчиков на складе с использованием ForkJoinPool. Задача по переносу товаров разделяется на подзадачи, которые выполняются параллельно с соблюдением ограничения по весу.

```
package Lab_8;

import java.io.IOException;
import java.util.*;
import java.util.stream.Collectors;


public class Demoversion {
    public static void main(String[] args) throws IOException {
        DataManager dataManager = new DataManager();
        dataManager.registerDataProcessor(new Transform());
        dataManager.registerDataProcessor(new Filter());
        dataManager.registerDataProcessor(new Aggregate());

        dataManager.loadData("File.txt");
        dataManager.processData();
        dataManager.saveData("Answer.txt");
    }
}


class Transform {
    @DataProcessor
    public List<String> transformData(List<String> data) {
        return data.stream()
                .map(String::toUpperCase)
                .collect(Collectors.toList());
    }
}

class Filter {
    @DataProcessor
    public List<String> filterData(List<String> data) {
        return data.stream()
                .filter(s -> s.startsWith("I"))
                .collect(Collectors.toList());
    }
}

class Aggregate {
    @DataProcessor
    public List<String> aggregateData(List<String> data) {
        return data.stream()
                .sorted()
```

```
                .distinct()
                .collect(Collectors.toList());
    }
}
```

Элемент 1 — Код программы 1

```
C:\Users\proto\.jdks\openjdk-25\bin\java.exe "-javaagent:E:\Programs\I
Применяется обработчик: transformData
Результат: [ITEM, APPLE, ICE, ITEM, BANANA, INK, IMPORTANT, ITEM]
Применяется обработчик: filterData
Результат: [ITEM, ICE, ITEM, INK, IMPORTANT, ITEM]
Применяется обработчик: aggregateData
Результат: [ICE, IMPORTANT, INK, ITEM]

Process finished with exit code 0
```

Элемент 2 — Результат программы

```
package Lab_8;


import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.*;
import java.util.concurrent.*;

public class DataManager {

    private List<Object> processors = new CopyOnWriteArrayList<>();
    private List<String> data;

    public void loadData(String source) throws IOException {
        Path path = Paths.get(source);
        String content = Files.readString(path, StandardCharsets.UTF_8);
        data = Arrays.asList(content.split("\\s"));
    }

    public void processData() {
        for (Object processor : processors) {
            Arrays.stream(processor.getClass().getDeclaredMethods())
                    .filter(method ->
method.isAnnotationPresent(DataProcessor.class))
                    .forEach(method -> {
                        try {
                            System.out.println("Применяется обработчик: " +
method.getName());
                            data = (List<String>) method.invoke(processor,
data);
                            System.out.println("Результат: " + data);
                        } catch (Exception e) {
```

```
                                e.printStackTrace();
                            }
                        });
            }
        }

        public void saveData(String destination) throws IOException {
            Path path = Paths.get(destination);
            Files.write(path, data, StandardOpenOption.CREATE);
        }

        public void registerDataProcessor(Object processor) {
            processors.add(processor);
        }

}
```

Элемент 3 — Код программы 2

Заключение

В ходе выполнения лабораторной работы были успешно изучены и применены на практике основные механизмы многопоточного программирования в Java. Были реализованы программы, демонстрирующие:

- использование ExecutorService для создания пула потоков и параллельной обработки данных;
- создание и управление потоками через наследование от класса Thread;
- применение ForkJoinPool для рекурсивного разделения задач на подзадачи;
- синхронизацию работы потоков и объединение результатов.

Работа позволила закрепить навыки проектирования многопоточных приложений, понимание принципов параллельных вычислений и методов оптимизации производительности программ.

Github: https://github.com/Prototype721/Java